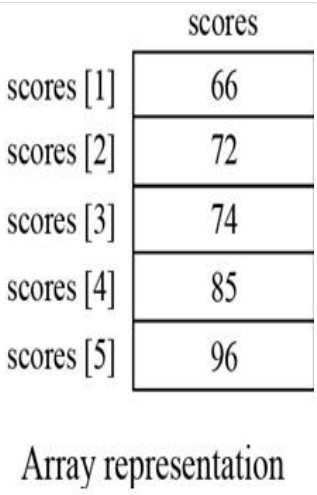
DATA STRUCTURE DETAILED NOTES FOR END TERM

DATA STRUCTURE : - A data structure defines a way of organizing all data items that considers not only the elements stored but also their relationship to each other. The term data structure is used to describe the way data is stored. For e.g. contact list on our mobile phones

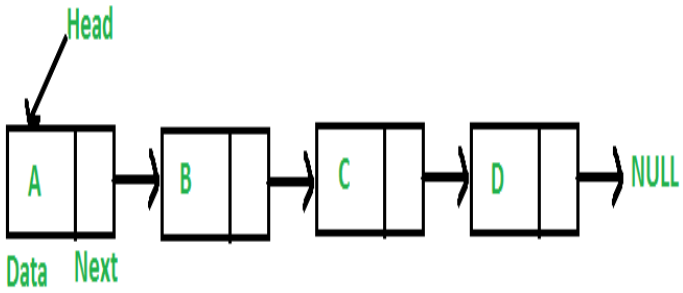
A data structure is said to be ***linear*** if its elements form a sequence or a linear list. The linear data structures like **an array, stacks, queues and linked lists** organize data in linear order. A data structure is said to be ***non linear*** if its elements form a hierarchical classification where, data items appear at various levels e.g. trees like heap , b-tree, avl tree etc

Primitive Data Structures are the basic data structures that directly operate upon the machine instructions. They have different representations on different computers. **Integers, floating point numbers, character constants, string constants and pointers** come under this category.

	Array	Linked List
Define	Array is a collection of elements having same data type with common name.	Linked list is an ordered collection of elements which are connected by l
Access	In array, elements can be accessed using index/subscript value, i.e. elements can be randomly accessed like arr[0], arr[3], etc. So array provides fast and **random access.**	In linked list, elements can't be accessed randomly but can be accessed only **sequentially**and accessing element takes **o(n) time**.
Memory Structure	In array, elements are stored in **consecutive** manner in memory.	In linked list, elements can be stored at any available place as address of node is stored in previous node.
Insertion & Deletion	Insertion & deletion takes more time in array as elements are stored in consecutive memory locations.	Insertion & deletion are fast & easy in linked list as only value of pointer is needed to change.
Memory Allocation	In array, memory is allocated at compile time i.e. **Static Memory Allocation**.	In linked list, memory is allocated at run time i.e . **Dynamic Memory Allocation.**
Types	Array can be **single dimensional**, two dimension or **multidimensional**.	Linked list can be **singly**, **doubly** or **circular** linked list.
Dependency	In array, each element is independent, no connection with previous element or with its location.	In Linked list, location or address of elements is stored in the link part of previous element/node.
Extra Space	In array, no pointers are used like linked list so no need of extra space in memory for pointer.	In linked list, adjacency between the elements are maintained using pointers or links, so pointers are used and for that extra memory space is needed.



Array representation



Non-primitive data structures are more complicated data structures and are derived from primitive data structures. They emphasize on grouping same or different data items with relationship between each data item**. Arrays, linked lists, trees and files come under this category**

Abstract Data Type (ADT): . An abstract data type in a theoreticalconstruct that consists of data as well as the operations to be performed on the data while hiding implementation.

For example, a stack is a typical abstract data type. Items stored in a stack can only be added and removed in certain order – the last item added is the first item removed. We call these operations, pushing and popping.

Time Complexity: The time needed by an algorithm expressed as a function of the size of a problem is called the **TIME COMPLEXITY** of the algorithm. The time complexity of a program is the amount of computer time it needs to run to completion

Space Complexity: The space complexity of a program is the amount of memory it needs to run to completion

Array advantages

1. Simple and easy to use
2. Any element can be accessed by using its index "I" .
3.Searching is easy

Disadvantages :- 1.Array is declared at compile time so its size once declared remains fixed throughout the program.

2. Insertion and deletion in array is difficult as all the elements are needed to be shifted.

Linked list advantages :-

a. Linked lists are dynamic data structures. i.e., they can grow or shrink during the execution of a program.
b. Linked lists have efficient memory utilization. Here, memory is not pre-allocated. Memory is allocated whenever it is required and it is de-allocated (removed) when it is no longer needed.
c. Insertion and Deletions are easier and efficient. Linked lists provide flexibility in inserting a data item at a specified position and deletion of the data item from the given position.
d. Many complex applications can be easily carried out with linked lists.

Disadvantages of linked lists:

1. It consumes more space because every node requires a additional pointer to store address of the next node.
2. Searching a particular element in list is difficult and also time consuming.

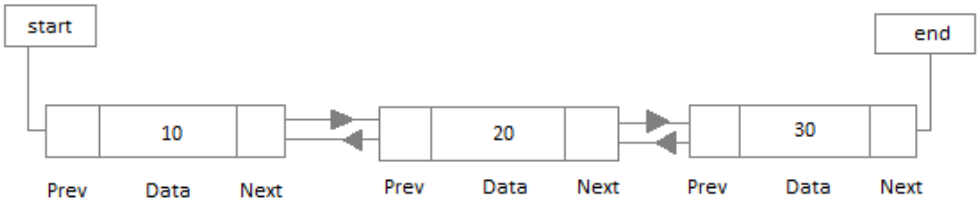
Types of Linked Lists:

Basically we can put linked lists into the following four items:

a. Single Linked List.
b. Double Linked List.
c. Circular Linked List.
d. Circular Double Linked List.

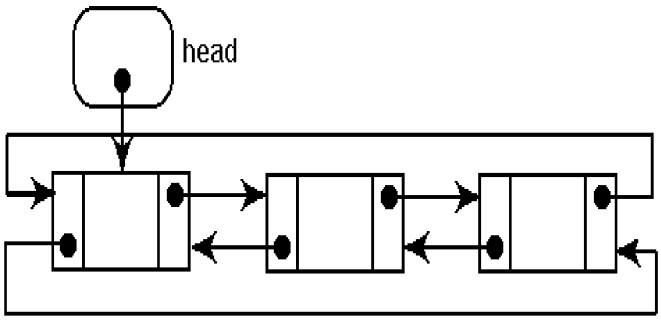
A single linked list is one in which all nodes are linked together in some sequential manner. Hence, it is also called as linear linked list.

A double linked list is one in which all nodes are linked together by multiple links which helps in accessing both the successor node (next node) and predecessor node (previous node) from any arbitrary node within the list



A circular linked list is one, which has no beginning and no end. A single linked list can be made a circular linked list by simply storing address of the very first node in the link field of the last node.

A circular double linked list is one, which has both the successor(next) pointer and predecessor (previous) pointer in the circular manner.



Doubly Linked Circular list

malloc() is a system function which allocates a block of memory in the "heap" and returns a pointer to the new block. It is used to allocate memory at RUN-TIME

free() is the opposite of malloc(), which de-allocates memory

Calloc () is also similar to malloc but it initializes every element to a value 0, Which is the only difference.

The basic operations in a single linked list are:

1. Creation.
2. Insertion.
3. Deletion.
4. Traversing

Creating a node for Single Linked List: Creating a singly linked list starts with creating a node. The information is stored in the memory, allocated by using the malloc() function.

```
node* getnode()
{
      node* newnode;
      newnode = (node *) malloc(sizeof(node)); printf("\n Enter data: ");
scanf("%d", &newnode -> data);
      newnode -> next = NULL;
      return newnode;
}
```

The function getnode(), is used for creating a node, after allocating memory for the structure of type node, the information for the item (i.e., data)

has to be read from the user, set next field to NULL and finally returns the address of the node.

Insertion of a Node
Inserting a node at the beginning:

The following steps are to be followed to insert a new node at the beginning of the list: Algortihm :

Get the new node using getnode().
newnode =getnode();

3. If the list is empty then *start = newnode.*

4.
5. If the list is not empty, follow the steps given below: newnode -> next = start;
start = newnode;

Figure 3.2.5 shows inserting a node into the single linked list at the beginning.

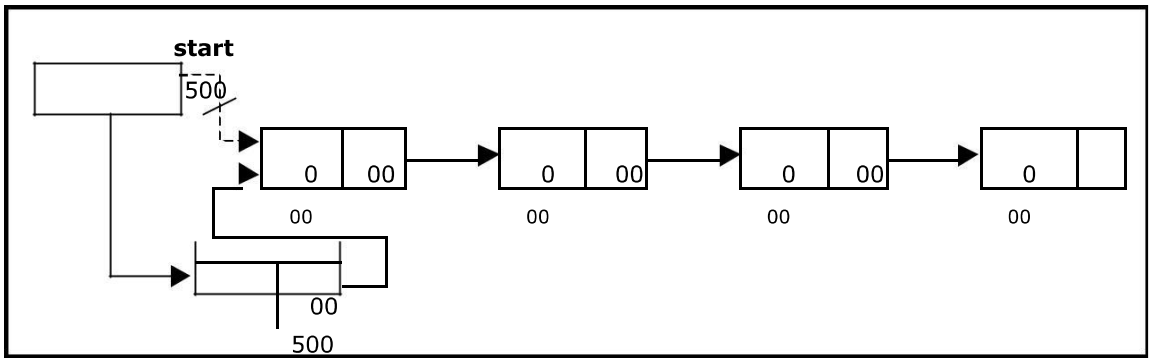


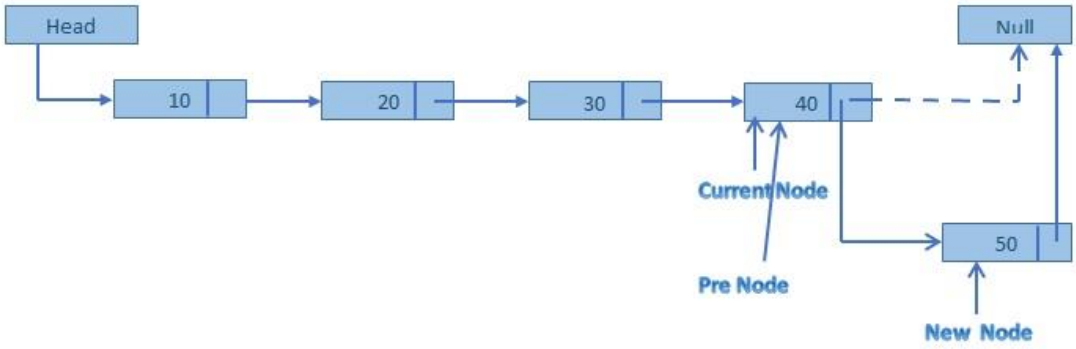
Figure . Inserting a node at the beginning

Inserting a node at the end:

The following steps are followed to insert a new node at the end of the list:

1. **Get the new node using getnode()**
newnode = getnode();

2. **If the list is empty then *start = newnode*.**
3. **If the list is not empty follow the steps given below:**
temp = start;
while(temp -> next != NULL)
temp = temp -> next;
temp -> next = newnode



Procedure For Inserting an element to linked list

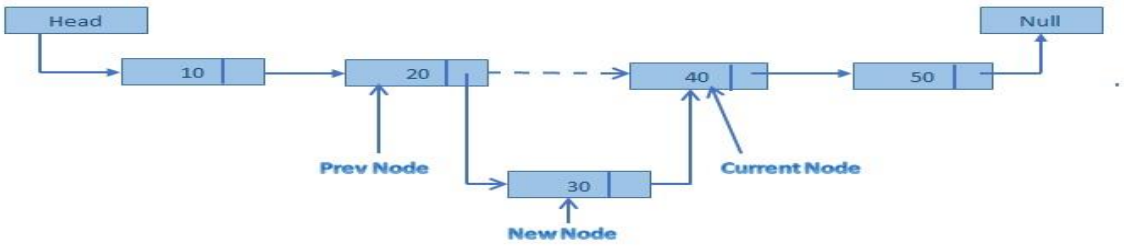
Step-1: Get the value for NEW node to be added to the list and its position

Step-2: Create a NEW, empty node by calling malloc(). If malloc() returns no error then go to step-3 or else say "Memory shortage".

Step-3:insert the data value inside the NEW node's data field.

Step-4: Add this NEW node at the desired position (pointed by the "location") in the LIST.

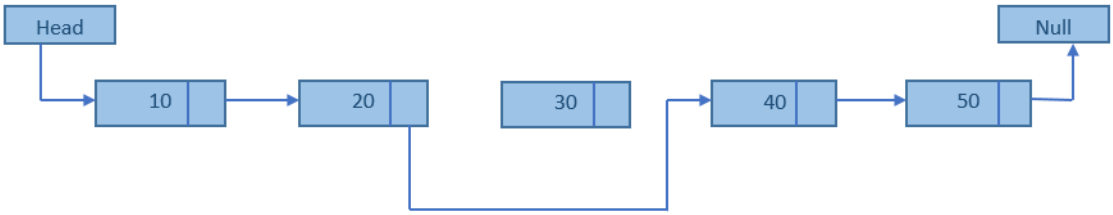
Step-5: Go to step-1 till you have more values to be added to the LIST.



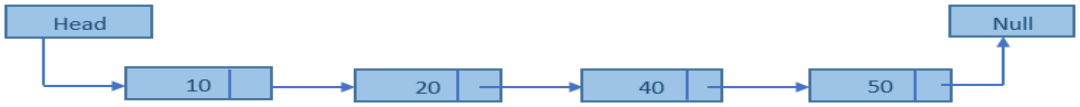
Deletion in Linked List

List before deletion

At the time of deletion



After the deletion



Procedure :

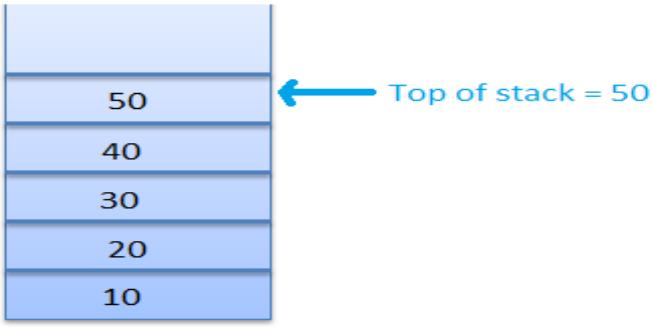
1.create a temporary pointer
2. store the address of node 40 from 30
Node-next = temp

3. store the address of node 40(present in temp) to the next of node 20
Node->next=temp

Concept of Stack

A Stack is data structure in which addition of new element or deletion of existing element always takes place at a same end. This end is known as the top of the stack. That means that it is possible to remove elements from a stack in reverse order from the insertion of elements into the stack.

One other way of describing the stack is as a last in, first out (LIFO) abstract data type and linear data structure.



Structure of Stack

Operations on Stack

The stack is basically performed two operations PUSH and POP.
Push and pop are the operations that are provided for insertion of an element into the stack and the removal of an element from the stack, respectively.
PUSH:- PUSH operation performed for the adding item to the stack.
POP:- POP operation performed for removing an item from a stack.

- **Applications of stacks:**

- Stack is used by compilers to check for balancing of parentheses, brackets and braces.
- Stack is used to evaluate a postfix expression.
- Stack is used to convert an infix expression into postfix/prefix form.
- In recursion, all intermediate arguments and return values are stored on the processor's stack.
- During a function call the return address and arguments are pushed onto a stack and on return they are popped off.

Infix to Postfix Conversion

Procedure for Postfix Conversion

1. Scan the Infix string from left to right.
2. Initialize an empty stack.
3. If the scanned character is an operand, add it to the Postfix string.
4. If the scanned character is an operator and if the stack is empty push the character to stack.
5. If the scanned character is an Operator and the stack is not empty, compare the precedence of the character with the element on top of the stack.
6. If top Stack has higher precedence over the scanned character pop the stack else push the scanned character to stack. Repeat this step until the stack is not empty and top Stack has precedence over the character.
7. Repeat 4 and 5 steps till all the characters are scanned.
8. After all characters are scanned, we have to add any character that the stack may have to the Postfix string.
9. If stack is not empty add top Stack to Postfix string and Pop the stack.
10. Repeat this step as long as stack is not empty

EXAMPLE:

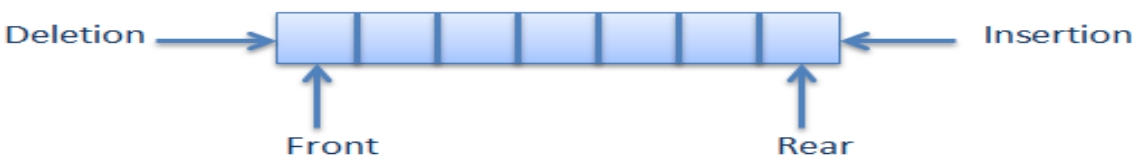
```
A+(B*C-(D/E-F)*G)*H
```

Stack	Input	Output
Empty	A+(B*C-(D/E-F)*G)*H	-
Empty	+(B*C-(D/E-F)*G)*H	A
+	(B*C-(D/E-F)*G)*H	A
+(	B*C-(D/E-F)*G)*H	A
+(	*C-(D/E-F)*G)*H	AB
+(*	C-(D/E-F)*G)*H	AB
+(*	-(D/E-F)*G)*H	ABC
+(-	(D/E-F)*G)*H	ABC*
+(-(	D/E-F)*G)*H	ABC*
+(-(	/E-F)*G)*H	ABC*D
+(-(/	E-F)*G)*H	ABC*D

+(-(/	-F)*G)*H	ABC*DE
+(-(-	F)*G)*H	ABC*DE/
+(-(-	F)*G)*H	ABC*DE/
+(-(-	)*G)*H	ABC*DE/F
+(-	*G)*H	ABC*DE/F-
+(-*	G)*H	ABC*DE/F-
+(-*	)*H	ABC*DE/F-G
+	*H	ABC*DE/F-G*-
+*	H	ABC*DE/F-G*-
+*	End	ABC*DE/F-G*-H
Empty	End	ABC*DE/F-G*-H*+

Queues

Queues is a kind of abstract data type where items are inserted one end (rear end) known as **enqueue** operation and deteted from the other end(front end) known as **dequeue** operation. This makes the queue a **First-In-First-Out (FIFO)** data structure.

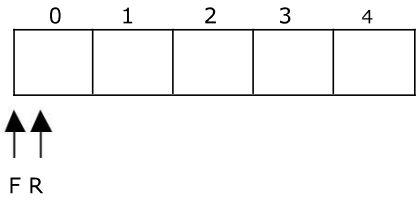


Operation on Queues

Operation	Description
initialize()	Initializes a queue by adding the value of rear and font to -1.
enqueue()	Insert an element at the rear end of the queue.
dequeue()	Deletes the front element and return the same.
empty()	It returns true(1) if the queue is empty and return false(0) if the queue is not empty.
full()	It returns true(1) if the queue is full and return false(0) if the queue is not full.

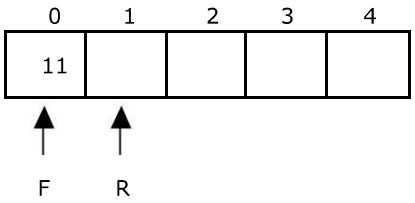
Representation of Queue:

Let us consider a queue, which can hold maximum of five elements. Initially the queue is empty.



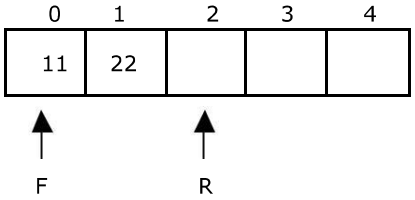
Que u e E mpt y
F RO NT = REA R = 0

Now, insert 11 to the queue. Then queue status will be:



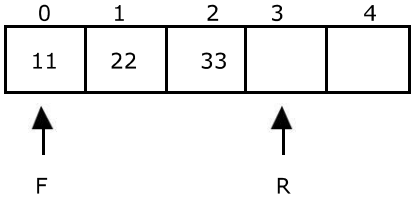
REA R = REA R + 1 = 1
F RO NT = 0

Next, insert 22 to the queue. Then the queue status is:



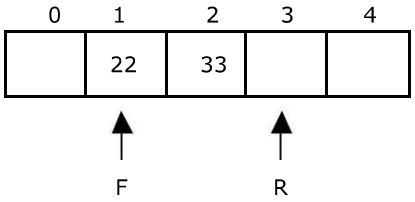
REA R = REA R + 1 = 2
F RO NT = 0

Again insert another element 33 to the queue. The status of the queue is:



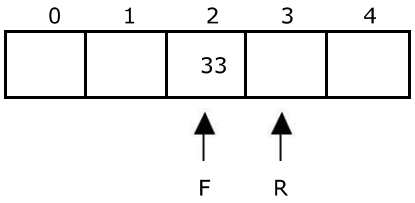
REA R = REA R + 1 = 3
F RO NT = 0

Now, delete an element. The element deleted is the element at the front of the queue. So the status of the queue is:



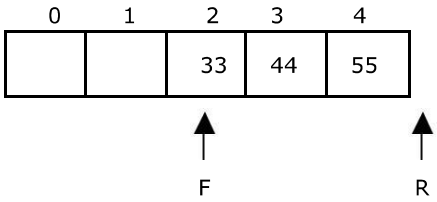
REA R = 3
F RO NT = F R O NT + 1 = 1

Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The queue status is as follows:



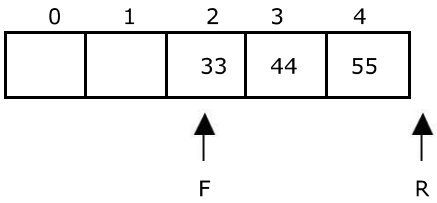
REA R = 3
F RO NT = F R O NT + 1 = 2

Now, insert new elements 44 and 55 into the queue. The queue status is:



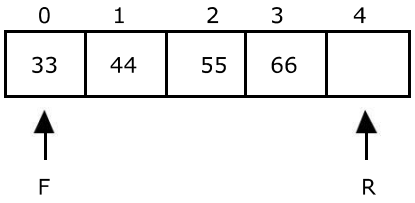
REA R = 5
F RO NT = 2

Next insert another element, say 66 to the queue. We cannot insert 66 to the queue as the rear crossed the maximum size of the queue (i.e., 5). There will be queue full signal. The queue status is as follows:



REA R = 5
F RO NT = 2

Now it is not possible to insert an element 66 even though there are two vacant positions in the linear queue. To over come this problem the elements of the queue are to be shifted towards the beginning of the queue so that it creates vacant position at the rear end. Then the FRONT and REAR are to be adjusted properly. The element 66 can be inserted at the rear end. After this operation, the queue status is as follows:



REA R = 4
F RO NT = 0

This difficulty can overcome if we treat queue position with index 0 as a position that comes after position with index 4 i.e., we treat the queue as a **circular queue.**

Applications of Queue:

- It is used to schedule the jobs to be processed by the CPU.
- When multiple users send print jobs to a printer, each printing job is kept in the printing queue. Then the printer prints those jobs according to first in first out (FIFO) basis.
- Breadth first search uses a queue data structure to find an element from a graph.

Dequeue :- data can be inserted or deleted from any end

There are two variations of deque. They are:

An Input restricted dequeue is a deque, which allows insertions at one end but allows deletions at both ends of the list.

An output restricted deque is a deque, which allows deletions at one end but allows insertions at both ends of the list.

Priority Queue:A priority queue is a collection of elements such that each element has been assigned a priority An element of higher priority is processed before any element of lower priority.Two elements with same priority are processed according to the order in which they were added to the queue

TREES

Leaf node

A node with no children is called a *leaf* (or *external node*). A node which is not a leaf is called an *internal node*.

SiblingsThe children of the same parent are called siblings.

Ancestor and Descendent If there is a path from node A to node B, then A is called an ancestor of B and *B* is called a descendent of A.

Level The level of the node refers to its distance from the root. The root of the tree has level O, and the level of any other node in the tree is one more than the level of its parent. *The maximum number of nodes at any level is* 2^n.

Height The maximum level in a tree determines its height. The height of a node in a tree is the length of a longest path from the node to a leaf. The term depth is also used to denote height of the tree

6. If *h* = height of a binary tree, then

   Maximum number of leaves = 2^h

   Maximum number of nodes = $2^{h+1} - 1$

Full Binary tree:

All non leaf nodes of a full binary tree have two children, and the leaf nodes have no children.

Complete binary tree A complete binary tree of height h looks like a full binary tree down to level h-1, and the last level level h is filled from left to right.

TREE TRAVERAL TECHNIQUES

7. **INORDER** : Visit the left subtree,
8. Visit the root.
9. Visit the right subtree,

Preorder Traversal:

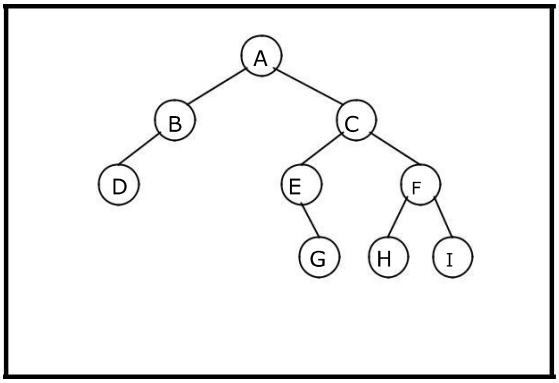
10. Visit the root.
11. Visit the left subtree
12. Visit the right subtree

Postorder Traversal:

13. Visit the left subtree,.
14. Visit the right subtree
15. Visit the root.

Example 1:

Traverse the following binary tree in pre, post, inorder and level order.



Bina ry T re e

16. Preo rder traversal
yields: A, B, D, C , E, G , F , H, I

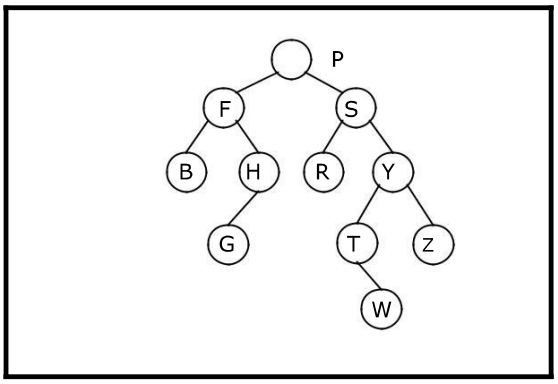
17. Posto rder traversal
yields: D, B, G , E, H, I, F , C , A

18. Ino rder traversal
yields: D, B, A, E, G , C , H, F , I

Pre, P o st, Inorder a nd lev e l order T rav ers ing

Example 2:

Traverse the following binary tree in pre, post, inorder and level order.



19. Preo rder traversal yields:
P, F , B, H, G , S, R, Y, T, W, Z

20. Posto rder traversal yields:
B, G , H, F , R, W, T, Z, Y, S, P

21. Ino rder traversal yields:
B, F , G , H, P, R, S, T, W, Y, Z